

A Minor Project Report on

RESUME RANKING SYSTEM USING S-BERT AND WORD2VEC



in partial fulfillment for the award of the degree of
Bachelor of Engineering in Information Technology

Submitted By

Alisha Singh	[23120161]
Prabin Rimal	[23120177]
Puja Regmi	[23120180]
Shuvam Khadka	[23120195]

Submitted To

Department of Software and IT Engineering
Everest Engineering College
Sanepa, Lalitpur-02

July, 2026

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Pokhara University and Everest Engineering College for providing us with the academic environment and opportunity to carry out our third-year minor project, “Resume Ranking System Using Hybrid SBERT-Word2Vec Approach,” as a partial fulfillment of the requirements for the Bachelor’s degree. We are deeply thankful to our project supervisor, Er. Mamata Tharu, for her continuous guidance, encouragement, and valuable feedback throughout the project. Her support and expertise greatly contributed to the successful development of this system. We would also like to extend our sincere appreciation to the Head of Department, Er. Pradip Paudel, for his academic guidance and encouragement. Our heartfelt thanks go to the management team of Everest Engineering College for providing the necessary resources and supportive academic environment. We are grateful to all faculty members and the project evaluation panel for their valuable suggestions and constructive feedback. We would like to thank our team members for their cooperation, dedication, and teamwork throughout the project. Finally, we express our sincere gratitude to our family and friends for their constant support and motivation.

ABSTRACT

The increasing volume of digital job applications has made resume screening a challenging and time-consuming task for Human Resource (HR) departments. Traditional Applicant Tracking Systems (ATS) relied primarily on keyword-based matching, which often failed to capture the semantic meaning of resumes and job descriptions and was susceptible to keyword stuffing. To address these limitations, we developed a context-aware resume ranking framework that leverages semantic embeddings to improve candidate shortlisting. We utilized the all-MiniLM-L6-v2 Sentence-BERT (S-BERT) model, populated using approximately 1,429 resumes sourced combinedly from different sites across Google, and Kaggle public datasets. The model generates 384-dimensional semantic embeddings for each resume, which are stored and subsequently ranked against a new job description (JD) using cosine similarity to retrieve the top-10 most relevant candidates. In parallel, a Word2Vec model was trained on the same corpus to produce document-level embeddings through average word vector aggregation, which were similarly ranked using cosine similarity. The S-BERT and Word2Vec embedding pipelines were built, and individual similarity rankings from each model were computed and analyzed. Preliminary similarity scores from both models are presented and compared. We ensembled hybrid scoring strategy combining their outputs, and integrated into a web application for interactive display.

Keywords: *Applicant Tracking System, Cosine Similarity, Resume Ranking, S-BERT, Semantic Embedding, Word2Vec.*

TABLE OF CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT	ii
LIST OF ABBREVIATIONS	vii
1 INTRODUCTION	1
1.1 Background	1
1.2 Statement of Problem	2
1.3 Objective	2
1.4 Scope	2
1.5 Applications	3
2 LITERATURE REVIEW	4
3 METHODOLOGY	7
3.1 Prediction Process Flow	7
3.2 System Design	8
3.3 Data Aquisition	9
3.4 Data Preprocessing	10
3.4.1 Text Cleaning	10
3.4.2 Lowercasing	10
3.4.3 Tokenization	11
3.4.4 Stopword Removal	11
3.4.5 Lemmatization	11
3.4.6 Normalization and Final Cleaning	12
3.5 Feature Engineering	13
3.5.1 S-BERT Branch	13
3.5.2 Word2Vec Branch	14
3.6 Similarity Calculation	15
3.7 Candidate Ranking	16
3.8 Final Output	16
3.9 Technology Stack	16
4 RESULT AND ANALYSIS	17
4.1 Precision@k	18
4.2 Mean Reciprocal Rank (MRR)	18
4.3 Normalized Discounted Cumulative Gain (NDCG@k)	19

4.4	Cosine Similarity Score	19
4.5	Technical Analysis & Interpretation	19
4.6	Overall Evaluation	20
5	CONCLUSION AND FUTURE WORK	21
5.1	Conclusion	21
5.2	Future Work	21
	REFERENCES	22
	APPENDIX	23
A	Word2Vec Word Cloud	1
B	Web UI Images	2
C	Metric Performance Images	5

LIST OF TABLES

2.1	Summary of Literature Review	6
3.1	Dataset Categories and Resume Counts	9
3.2	Technology Stack	16
4.1	Retrieval Performance Comparison	18

LIST OF FIGURES

3.1	Prediction Process Flow	7
3.2	Block Diagram	9
3.3	Data Preprocessing Pipeline	12
3.4	Preprocessing for S-BERT	12
3.5	Preprocessing for Word2Vec	13
3.6	Sentence-BERT Architecture	13
3.7	Word2Vec architecture	15
4.1	Comparison Analysis of Individual and Hybrid Score	17
A.1	Word2Vec Wordcloud	1
A.2	Word Cloud for DJango Resume	1
B.1	Providing JD Text	2
B.2	Ranked Available Resumes For Given JD Text	2
B.3	Uploading PDFs	3
B.4	Ranked Available Resumes For Given JD PDF	3
B.5	Candidate Sorting	4
C.1	Performance Metrics	5

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ATS	Applicant Tracking System
BERT	Bidirectional Encoder Representations from Transformers
CSV	Comma-Separated Values
HR	Human Resources
HTML	HyperText Markup Language
JD	Job Description
KNN	K-Nearest Neighbors
LDA	Latent Dirichlet Allocation
ML	Machine Learning
MRR	Mean Reciprocal Rank
NDCG	Normalized Discounted Cumulative Gain
NLP	Natural Language Processing
NER	Named Entity Recognition
PDF	Portable Document Format
S-BERT	Sentence Bidirectional Encoder Representations from Transformers
TF-IDF	Term Frequency–Inverse Document Frequency
Word2Vec	Word to Vector

CHAPTER 1

INTRODUCTION

1.1 Background

The recruitment process is a critical organizational function, yet it has become increasingly burdensome as the volume of digital job applications continues to grow. For a single job posting, HR departments routinely receive hundreds of resumes, rendering manual screening time-consuming, costly, and susceptible to unconscious bias. Resumes represent the primary medium of candidate evaluation, encapsulating skills, education, and experience, but their unstructured and heterogeneous nature makes systematic analysis particularly difficult.

Traditional Applicant Tracking Systems (ATS) depend on keyword matching and rule-based filtering [1]. Such methods are fundamentally limited because they cannot model semantic meaning, and suitably qualified candidates are frequently rejected due to vocabulary mismatches. Subsequent machine learning approaches, such as TF-IDF combined with KNN, improved retrieval effectiveness but continued to fall short in capturing the deep contextual relationships that exist between job requirements and candidate profiles.

Distributed word representations such as Word2Vec, enabled systems to capture semantic similarity between terms even when exact keyword overlap is absent. When applied to resume ranking, averaging word vectors across a document produces a compact, semantically informed representation of candidate profiles that can be compared efficiently against a job description using cosine similarity [7].

Transformer-based language models such as BERT have further advanced natural language understanding by generating rich, context-sensitive embeddings. Sentence-BERT (S-BERT), introduced by Reimers and Gurevych, extends BERT with a siamese network architecture to produce fixed-length sentence embeddings optimized for semantic similarity tasks [12]. When applied to bulk resume corpora, S-BERT embeddings can be pre-computed and stored, enabling near-instant ranked retrieval against a new job description at inference time. This project exploits this property by pre-encoding approximately 1,429 resumes [16] with S-BERT and with Word2Vec, then compare both ranking strategies and, combine them into a hybrid ensemble scoring and ranking.

1.2 Statement of Problem

Despite the proliferation of digital recruitment tools, several critical shortcomings persist in existing approaches:

- Manual resume screening is time-consuming, inconsistent, and prone to bias.
- Keyword-based ATS systems cannot model semantic meaning, resulting in poor candidate-job fit assessments.
- Relevant candidates are frequently overlooked due to vocabulary mismatches between resumes and job descriptions.
- Single-model approaches (TF-IDF, Word2Vec, or BERT alone) each have distinct limitations that a hybrid strategy could mitigate.
- No efficient, scalable system currently exists that combines SBERT semantic understanding with Word2Vec embeddings and compares them objectively for resume ranking.

1.3 Objective

To develop a hybrid S-BERT and Word2Vec resume ranking system via weighted ensemble scoring for high-precision shortlisting.

1.4 Scope

The system scope encompasses the following functionalities:

- Cleaning and standardizing approximately 1,429 raw resumes from various sites of Google, and Kaggle for consistent input processing.
- Generating and storing 384-dimensional semantic embeddings for each resume using the all-MiniLM-L6-v2 S-BERT model.
- Generating and storing averaged Word2Vec document vectors for each resume using a Gensim Word2Vec model.
- Ranking the top-10 candidates against a new job description using cosine similarity for each model independently.
- Presenting and comparing individual similarity score tables for both models.
- Formal comparative evaluation (NDCG@K, MRR), hybrid ensemble scoring, and integration of ranked candidates into a web interface for interactive result display.

1.5 Applications

This hybrid resume ranking framework is engineered for versatile deployment across several key industries.

- Enterprise HR departments can automate high-volume resume screening to save time and reduce manual bias.
- Staffing agencies can instantly match client job descriptions across twenty-four distinct professional domains.
- Job boards can deploy the framework to recommend positions using semantic alignment rather than keywords.
- Large corporations can utilize the system to match internal employee profiles with new project vacancies.
- University placement cells can quickly rank student resumes against corporate hiring criteria during recruitment drives.

CHAPTER 2

LITERATURE REVIEW

Traditional recruitment processes rely heavily on manual resume screening and keyword-based ATS systems, which frequently fail to interpret the contextual meaning of candidate qualifications and job requirements. As the volume of applications has grown, researchers have explored machine learning and natural language processing (NLP) to automate resume parsing, candidate matching, and ranking. This chapter reviews the key works that motivate and contextualize the present project.

[1] proposed a methodology for resume parsing and job domain prediction that automatically extracts candidate information and classifies resumes into relevant categories using structured NLP pipeline. While effective for domain classification, their system relied on keyword extraction and lacked deeper semantic understanding. [2] similarly developed a machine learning-based resume recommendation system to automate candidate selection and improve recruitment efficiency, demonstrating that automation could significantly reduce manual effort. [3] introduced automated resume ranking software grounded in keyword matching, though useful for reducing screening time, the system demonstrated limited semantic understanding and was sensitive to lexical variation.

[4] enhanced recruitment automation further with a stacked machine learning model that combined multiple classifiers, achieving improved screening performance through ensemble methods, a concept that directly motivates the hybrid scoring strategy proposed in this project. [5] proposed a hybrid job and resume matching framework integrating multiple recommendation techniques, showing that combining complementary matching strategies consistently outperformed single-model approaches. [6] provided an early machine learning-based resume ranking design, validating the general feasibility of automated candidate ordering.

The limitations of purely keyword-based and shallow ML approaches led researchers to explore richer text representations. [7] introduced Word2Vec, a neural word embedding model that learns dense vector representations from large corpora such that semantically related words cluster in the vector space. When word vectors within a document are averaged, the resulting document vector captures broader semantic content and enables meaningful cosine similarity comparisons between resumes and job descriptions, a technique that forms one branch of the present system. [8] demonstrated that hybrid resume parsing approaches that combine multiple NLP signals significantly improved information extraction from unstructured resumes. [9] confirmed through empirical evaluation that

TF-IDF combined with cosine similarity provides a strong baseline for document-level similarity, but noted that its reliance on surface-level term frequency limits contextual understanding.

The advent of transformer-based language models opened a new frontier in resume ranking. [10] introduced a BERT-based resume parsing framework capable of extracting contextual meaning, significantly outperforming traditional NLP methods in both extraction accuracy and semantic matching. [11] applied Latent Dirichlet Allocation (LDA) together with NLP techniques for semantic analysis of resume content, highlighting that topic-level representations can complement word-level matching. [12] introduced Sentence-BERT (S-BERT), a siamese network-based adaptation of BERT that produces semantically meaningful fixed-length sentence embeddings optimized for similarity tasks. S-BERT overcomes a key limitation of the original BERT model, the prohibitive cost of pairwise inference, by allowing embeddings to be pre-computed and stored offline, making it practical for ranking large resume corpora against a new job description at inference time. This property makes S-BERT the primary embedding backbone of the present system.

Recent work has continued to refine candidate-job matching in applied settings. [13] developed a machine learning-based resume optimization model showing that targeted feature engineering can enhance matching performance even with compact representations. [14] proposed an enhanced KNN algorithm integrated with BERT embeddings for resume parsing, demonstrating that combining transformer representations with retrieval-based ranking improved contextual understanding compared to either approach alone. [15] further showed that NLP-driven AI systems could efficiently scale candidate screening to large applicant pools while maintaining ranking accuracy, reinforcing the case for semantic embedding approaches in enterprise recruitment contexts.

A survey of the existing literature reveals two persistent gaps. First, although Word2Vec and S-BERT have individually been applied to recruitment tasks, very few studies provide a direct, controlled comparison of their ranking outputs on the same corpus under identical evaluation conditions. Second, while ensemble and hybrid scoring methods have been shown to outperform single models in classification settings, their application to semantic resume ranking has not been thoroughly studied. The present project addresses both gaps: it pre-encodes 1,429 real-world resumes from [16] using both S-BERT and Word2Vec, ranks the top-10 candidates against a test job description independently using each model, and presents their individual cosine similarity scores for direct comparison. Future work will formalize this comparison using NDCG@K and MRR, and introduce a hybrid ensemble score to evaluate whether combining both models yields further improvements.

Table 2.1: Summary of Literature Review

Ref.	Method / Model	Key Contribution	Limitation
[1]	NLP pipeline: resume parsing & domain classification	Extracts candidate info and classifies resumes into domains	Keyword-based; lacks deep semantic understanding
[2]	ML-based resume recommendation	Automates candidate selection, improving recruitment efficiency	Limited semantic matching capability
[3]	Automated ranking via keyword matching	Reduces manual screening time	Sensitive to lexical variation; limited semantics
[4]	Stacked ML (ensemble classifiers)	Improved screening via ensemble methods	High complexity from combining classifiers
[5]	Hybrid job-resume matching framework	Multi-strategy approach outperforms single-model baselines	Requires integrating multiple techniques
[6]	Early ML-based resume ranking	Validated feasibility of automated candidate ordering	Foundational design; limited scope
[7]	Word2Vec (neural word embeddings)	Dense vectors enabling semantic word clustering	Averaged vectors lose fine-grained context
[8]	Hybrid NLP resume parsing	Multi-signal NLP improves extraction from unstructured text	Increased pipeline complexity
[9]	TF-IDF + cosine similarity	Strong baseline for document-level similarity	Surface-level term frequency; limited context
[10]	BERT-based resume parsing	Outperforms traditional NLP in extraction and semantic matching	High computational cost of pairwise inference
[11]	LDA + NLP	Topic-level representations complement word-level matching	Topics may miss fine-grained semantics
[12]	Sentence-BERT (S-BERT)	Siamese network for semantic embeddings; enables offline pre-computation	Requires transformer training resources
[13]	ML-based resume optimization	Feature engineering improves matching with compact representations	Manual, task-specific feature engineering
[14]	Enhanced KNN + BERT embeddings	Transformer + retrieval hybrid improves context understanding	Increased complexity from hybridization
[15]	NLP-driven AI screening	Scales screening to large applicant pools with maintained accuracy	Scalability tested in specific enterprise contexts

CHAPTER 3

METHODOLOGY

A systematic and modular methodology has been followed to design, implement, and evaluate the hybrid resume ranking system. The pipeline is structured around two parallel embedding branches, S-BERT and Word2Vec, whose outputs are independently used for candidate ranking and will subsequently be combined into a hybrid scoring model.

3.1 Prediction Process Flow

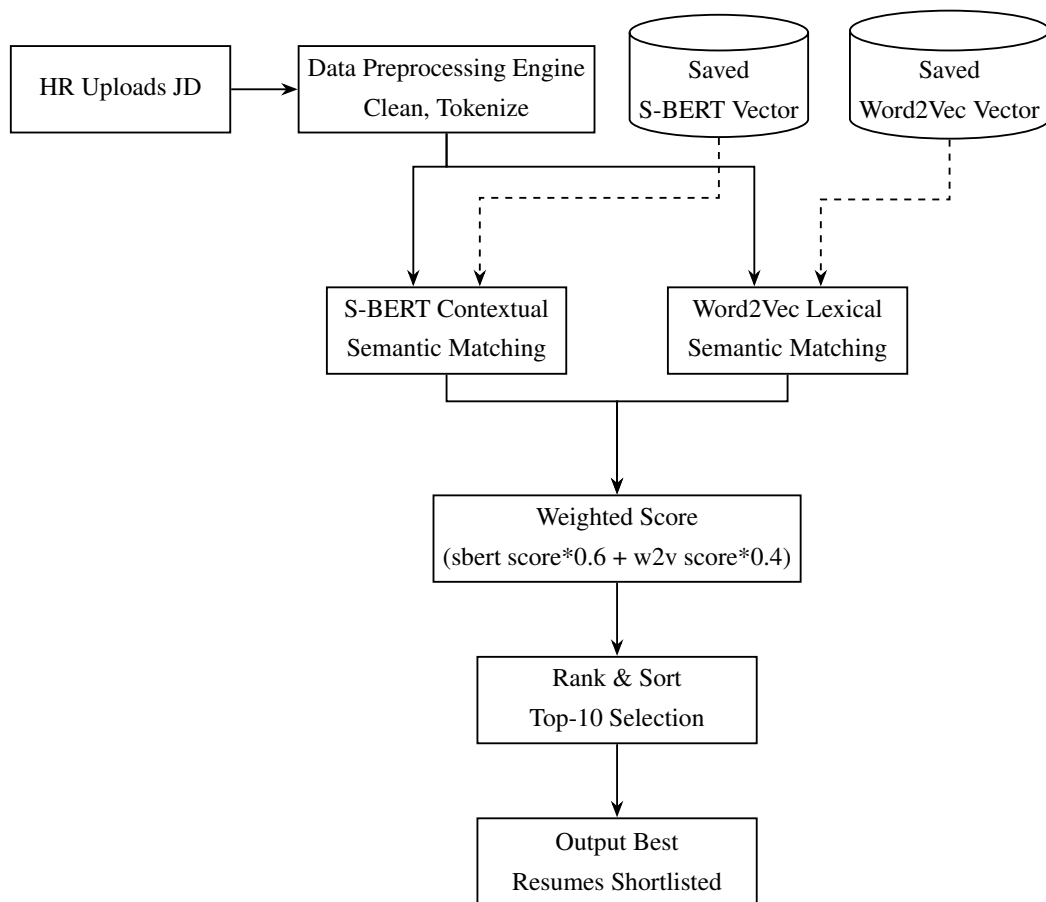


Figure 3.1: Prediction Process Flow

This figure shows the prediction process, which occurs whenever a HR wants to upload a JD. Not like the training process in Figure 3.2 below, this prediction process makes use of the pre-trained models for real-time prediction.

3.2 System Design

System design plays a crucial role in the development of an efficient and scalable resume ranking system. It defines how different components of the system interact with each other to process input data and generate meaningful outputs. A well-structured design ensures modularity, flexibility, and ease of future enhancements. The proposed Resume Ranking System follows a pipeline-based architecture, where the entire workflow is divided into multiple stages. Each stage is responsible for performing a specific task, and together they form a complete processing system.

- **Input Layer:** The system begins with the input layer, where data is collected from the user. Resume files are uploaded by candidates in PDF format and Job Description (JD) is provided by the recruiter. These inputs act as the primary data sources for the system. The design ensures that multiple resumes can be uploaded simultaneously, enabling batch processing.
- **Data Processing Layer:** In this layer, raw input data is processed and converted into a structured format. Resumes are parsed to extract textual content from PDF formats using document processing libraries. The extracted text is cleaned and normalized by removing special characters and symbols, converting text to lowercase, eliminating stopwords, performing tokenization and lemmatization.
- **Analysis and Matching Layer:** This is the most critical part of the system where intelligent decision-making takes place. The system uses advanced NLP models (such as BERT-based embeddings and Word2Vec) to compare resumes with job descriptions. Instead of relying only on keywords, it understands the contextual meaning of the text. Multiple parameters are combined to generate a final score. This combined score is used to evaluate the suitability of each candidate.

Semantic Matching (NLP-Based): Semantic matching addresses the limitations of keyword-based approaches by understanding the meaning of text. The system uses transformer-based models to convert text into vector representations (embeddings), measure similarity using cosine similarity (e.g., “Machine Learning”, “ML”, “Predictive Modeling”). All of these can be recognized as related concepts.

- **Ranking and Output Generation:** Finally, candidates are ranked based on their scores calculated by ensembling both model similarity scores using weighted sum approach.

$$Hybrid_Score = 0.60 * sbert_score + 0.40 * w2v_score \quad (3.1)$$

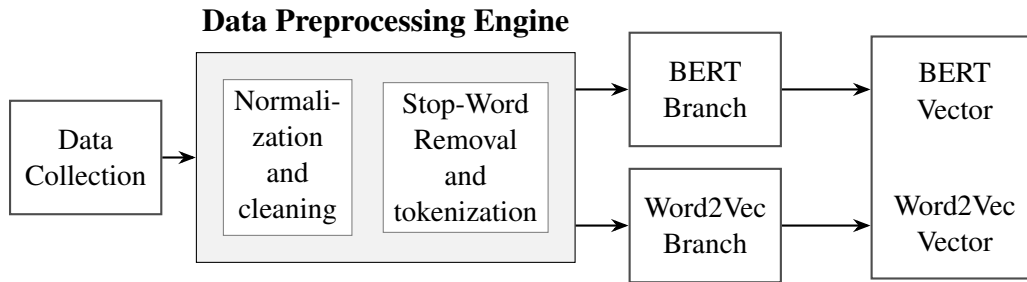


Figure 3.2: Block Diagram

3.3 Data Aquisition

The dataset used to populate database in this project was sourced from Google Dorking, and Kaggle which consists of approximately 1,429 resumes in PDF format, spanning 23 domains including ML Engineer, Python Developer, Data Science, HR, Designer, and others. This corpus provides a broad and realistic representation of candidate profiles against which job descriptions can be matched.

Table 3.1: Dataset Categories and Resume Counts

Categories	Counts
AI ML	119
BI	22
Blockchain	42
Cloud Solutions Architect	24
Cybersecurity Analyst	17
DESIGNER	107
DIGITAL MEDIA	96
DOT NET	66
Data Science	83
DevOps	80
Full Stack Developer	84
Game Developer	48
HR	110
IT Specialist	121
IoT Engineer	13
MERN Stack Developer	55
Mobile App Developer	31
Network Engineer	35
Python Developer	37

Categories	Counts
Robotics Engineer	61
Software Engineer	102
Systems Administrator	37
Technology Consultant	39
Total	1429

The HR supplies a single job description at inference time. Then the similarity computation is done between JD and all the available resumes and best matching candidates were shortlisted.

3.4 Data Preprocessing

All resumes and job descriptions are subjected to a unified preprocessing pipeline to improve input quality and consistency. Raw text is first extracted from PDF files using the PDFPlumber library. Then the preprocessing stage consists of sequential steps applied across both the SBERT and Word2Vec pipelines to transform raw resume text into clean, meaningful representations suitable for embedding and similarity computation.

The main objectives of data preprocessing in this system are to remove noise and irrelevant information, standardize text for uniform processing, improve the accuracy of feature extraction, enhance the performance of semantic similarity models, and reduce computational complexity. The preprocessing stage consists of several sequential steps:

3.4.1 Text Cleaning

Raw resume text and JD text extracted via PDFs undergoes foundational cleaning before any further processing. This includes removal of invisible Unicode control characters, PDF font artifacts, URLs, domain names, email addresses, phone numbers, physical addresses, and labeled personal fields such as Date of Birth and Nationality. Boilerplate content, including the candidate name block, noise phrases such as LinkedIn, GitHub, portfolio, and references available upon request, is also removed at this stage. Additionally, spaCy’s NER model is applied to the opening header segment to detect and strip PERSON, GPE, and LOC entities, ensuring candidate identity does not influence skill-based ranking.

3.4.2 Lowercasing

All text is converted to lowercase to ensure consistency across tokens. This prevents the same word from being treated as distinct terms due to case differences.

Example: “Python”, “PYTHON”, and “python” → “python”

3.4.3 Tokenization

The cleaned, lowercased text is split into individual word tokens. For the Word2Vec pipeline, tokenization is performed explicitly on the preprocessed text. For SBERT, tokenization is handled internally by the transformer’s subword tokenizer, which operates on full sentence input.

Example: “Experienced in Python and Machine Learning” → [“experienced”, “in”, “python”, “and”, “machine”, “learning”]

Tokenization helps in analyzing text at a granular level and is essential for NLP-based operations.

3.4.4 Stopword Removal

Common English words that carry no discriminative value for skill-based matching are removed from the token list. Words such as “and”, “the”, “is”, “in”, and “of” are filtered out to reduce noise and retain only meaningful, job-relevant terms.

Example: “Experienced in Python and Machine Learning” → [“experienced”, “in”, “python”, “machine”, “learning”]

This step reduces noise and improves processing efficiency.

3.4.5 Lemmatization

Words are reduced to their base or dictionary form to consolidate variations of the same term into a single token. This reduces vocabulary size and ensures that morphological differences do not fragment related concepts.

For example: “working”, “worked”, “works” → “work”

Lemmatization improves token matching consistency in the Word2Vec pipeline, where each unique token maps to a distinct vector.

3.4.6 Normalization and Final Cleaning

The processed text undergoes a final normalization pass to remove any residual noise. Bullet point symbols, special characters, repeated punctuation sequences, and empty brackets are stripped. Consecutive whitespace is collapsed into a single space. The result is a clean, standardized token sequence ready for vector embedding via Word2Vec or SBERT encoding.

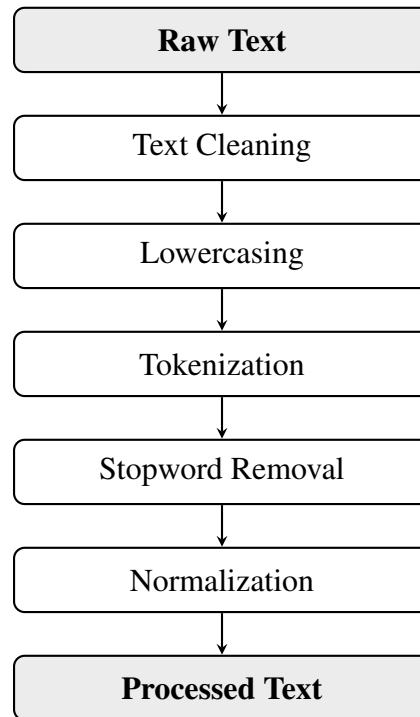


Figure 3.3: Data Preprocessing Pipeline

resume_id	Resume_str	clean_text_resume
0	0 Hieu-Thi Luong\ncontact@hieuthi.com https://...	experience in speech processing, deepfake dete...
1	1 bitbucket.org/mikkujacob github.com/mikhailj...	Education PhD Computer Science, Georgia Instit...
2	2 MERT KIRAY\nGörresstr. 3, 80798 München\n+4915...	Education TUM, Master of Science Informatics, ...
3	3 Adam Pardyl Email : adam@pardyl.com\nPhD Candi...	Education Jagiellonian University PhD candidat...
4	4 Karystinaios\nEmmanouil AI Researcher — Music ...	Profile Postdoctoral AI researcher with expert...

Figure 3.4: Preprocessing for S-BERT

resume_id		Resume_str	clean_text_resume
0	0	experience in speech processing, deepfake dete...	['experience', 'speech', 'process', 'deepfake'...
1	1	Education PhD Computer Science, Georgia Instit...	['education', 'phd', 'computer', 'science', 'g...
2	2	Education TUM, Master of Science Informatics, ...	['education', 'tum', 'master', 'science', 'inf...
3	3	Education Jagiellonian University PhD candidat...	['education', 'jagiellonian', 'university', 'p...
4	4	Profile Postdoctoral AI researcher with expert...	['profile', 'postdoctoral', 'researcher', 'exp...

Figure 3.5: Preprocessing for Word2Vec

3.5 Feature Engineering

Feature engineering in this system refers to the transformation of cleaned resume and JD text into numerical vector representations suitable for cosine similarity computation. Two independent branches are employed.

3.5.1 S-BERT Branch

To capture the exact meaning of text documents, the system implements a semantic embedding layer using Sentence-BERT (S-BERT). Sentence-BERT (SBERT) is a modification of the pre-trained BERT model that uses siamese network structures to derive semantically meaningful sentence embeddings suitable for cosine similarity comparison. Unlike vanilla BERT, which produces token-level representations, SBERT generates a single fixed-size vector for an entire text, making it efficient for large-scale semantic similarity tasks such as resume ranking [12].

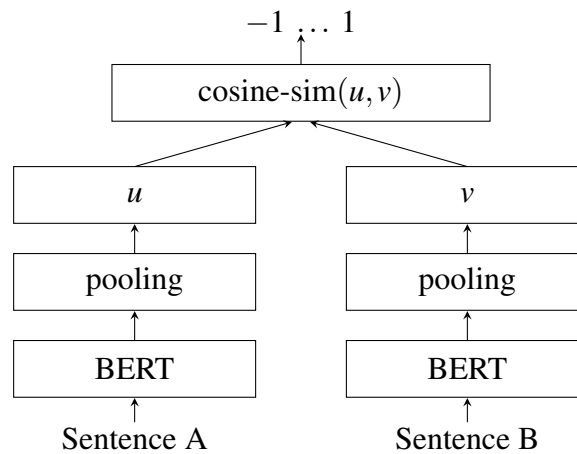


Figure 3.6: Sentence-BERT Architecture

- **Input Processing:** Raw resume text extracted from PDF is first lightly cleaned to preserve sentence structure, then tokenized using BERT's WordPiece tokenizer. Special tokens [CLS] and [SEP] are prepended and appended respectively. Since our chosen

model of S-BERT has a maximum input limit of 256 tokens and resumes often exceed this, a sliding window strategy is applied, the text is split into overlapping chunks of 200 words with a stride of 150 words. Each chunk is encoded independently.

- **Transformer Encoding:** Each tokenized chunk is passed through the pre-trained all-MiniLM-L6-v2 model, which consists of 6 transformer encoder layers with a hidden dimension of 384. Within each encoder layer, multi-head self-attention allows every token to attend to every other token in the sequence, producing context-aware token representations. A token such as Python therefore receives a different vector depending on whether the surrounding context is software development or biology.
- **Mean Pooling:** After the final encoder layer produces token-level representations then mean pooling is applied across all token vectors to obtain a single document-level embedding:

$$\vec{u} = \frac{1}{n} \sum_{i=1}^n T_i \quad (3.2)$$

For resumes processed with the sliding window, the pooled vectors from all chunks are further averaged into one final 384-dimensional vector $\vec{u}$ representing the entire resume.

Because the model acts as a Bi-Encoder, separate text sequences are processed completely independently of one another. The system pre-computes and caches these fixed-size semantic vectors for all documents in the database ahead of time. When a query is made, the system only needs to generate a vector for the new query string, instantly comparing it against the stored document vectors using fast, low-overhead geometric functions like Cosine Similarity.

3.5.2 Word2Vec Branch

To capture lexical features and semantic relations among words, the system utilizes the Word2Vec framework introduced by Google. Word2Vec is a shallow, two-layer neural network architecture trained to map words from a raw textual corpus into a low-dimensional continuous vector space [7]. The core architectural design operates under the distributional hypothesis, positioning words that share similar contextual environments in close geometric proximity within the vector space.

The pipeline evaluates embeddings through one of two structural configurations:

- **Continuous Bag-of-Words (CBOW):** The architecture minimizes loss by predicting a target token $W(t)$ from an aggregated window of surrounding context words, optimizing training efficiency for high-frequency terms.

- Skip-gram (SG): The architecture reverses the predictive objective by using a single center word to maximize log-probabilities of context tokens within a shifting frame window.

Upon network convergence, the output classification layer is discarded, and the underlying bottleneck weight matrix is retained as a static embedding lookup table. This maps each unique word token to a dense, real-valued feature vector capable of algebraic semantic compositionality (e.g., King - Man + Women = Queen).

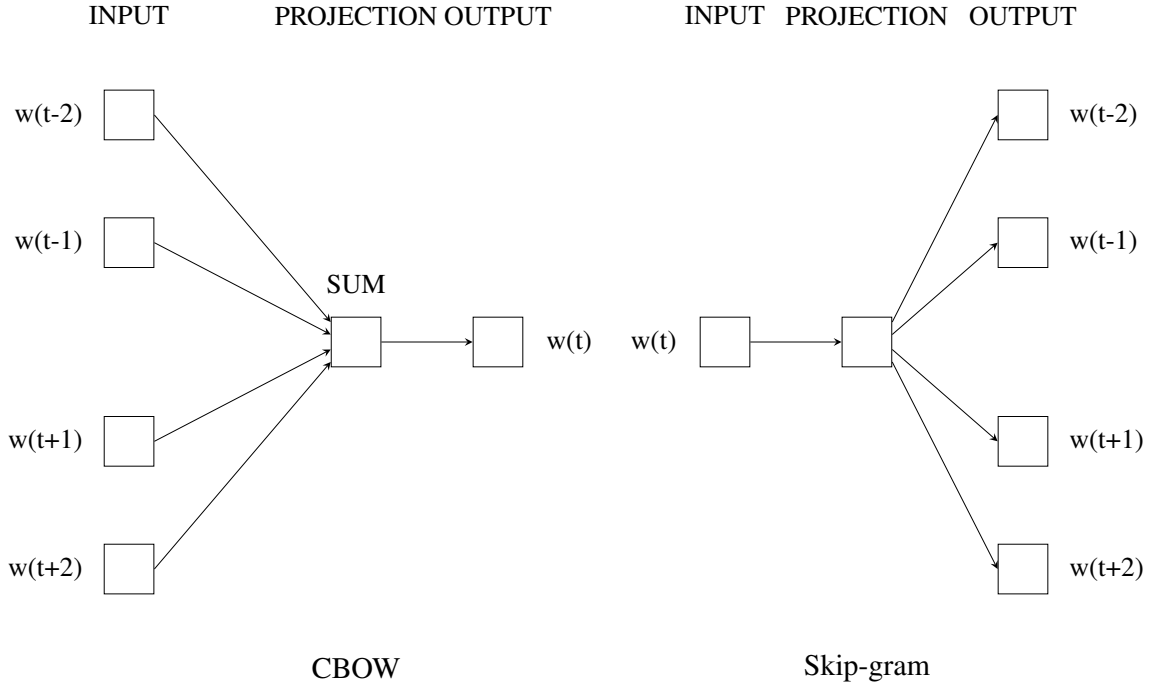


Figure 3.7: Word2Vec architecture

3.6 Similarity Calculation

The system calculates the similarity between the resume and the job for both branches using the embeddings generated by both contextually and lexically semantic models.

$$\text{sim}(V_{JD}, V_R) = \frac{V_{JD} \cdot V_R}{|V_{JD}| |V_R|} \quad (3.3)$$

where V_{JD} and V_R are the JD and resume embedding vectors, respectively.

Unlike the keyword search, the Semantic Engine successfully identified that a student knowing “Django” is a near-perfect match for a “Backend” role. This demonstrates the system’s ability to “understand” context, significantly improving shortlisting quality for technical roles where terminology varies.

3.7 Candidate Ranking

After computing cosine similarity scores between the JD and all resume vectors (independently for each model), the candidates are sorted in descending order of similarity. The top-10 candidates from each model's ranking are surfaced. This process is computationally efficient, since all resume embeddings are precomputed and stored, the inference-time cost is dominated by a single matrix-vector multiply operation, which completes in under one second for the entire corpus.

3.8 Final Output

Lastly, the system provides the results to the HR. It shorts the best matches at the top of the list and others in in descending order of the match score highlighting the top three matches with three different colors and remaining with same color.

3.9 Technology Stack

The following technologies were chosen to implement this system, based on their specific ability to support easy Web Development.

Table 3.2: Technology Stack

Components	Tools	Version/Model	Details
Core Backend	Python	3.13.5	Execution runtime and ML pipeline.
Web Server	Flask	3.1.0	RESTful API request handling.
Semantic Matching	sentence-transformers	all-miniLM-L2-V6	S-BERT contextual embeddings.
Lexical Embeddings	Gensim	W2V Skip-Gram	Word2Vec training and inference.
Similarity Calculation	scikit-learn	cosine similarity	High-dimensional cosine similarity.
Data Ingestion	PDFPlumber	0.11.10	Text extraction from resume PDFs.

CHAPTER 4

RESULT AND ANALYSIS

The empirical validation of the hybrid resume ranking system benchmarked the information retrieval efficacy of three distinct pipeline configurations, a locally trained Word2Vec model representing token-level features, a pre-trained SBERT transformer model representing dense semantic features, and the dual-embedding Hybrid Ensemble.



Figure 4.1: Comparison Analysis of Individual and Hybrid Score

To simulate high-friction, real-world deployment conditions and evaluate model robustness, an adversarial testing protocol was established. The system was challenged with an ad-hoc Job Description (JD) query for a “Django Backend Developer.” To thoroughly test the models’ capacity for deep conceptual inference rather than simple surface-level token matching, the key domain identifier “Python” was completely withheld from the query text. The requirement relied exclusively on secondary ecosystem indicators, including Django, ORM, REST APIs, relational databases, caching, and server-side logic.

The evaluation data pool comprised $N = 1,429$ multi-category candidate resumes. Objective ground-truth annotations were derived from the dataset’s validated category classifications to construct a binary relevance matrix R :

- Relevant (1): Resumes explicitly classified under the “Python Developer” domain.
- Irrelevant (0): Resumes belonging to any alternative engineering or non-technical classification (e.g., MERN Stack Developer, Database Administrator, Web Designer, Project Manager).

System retrieval effectiveness was measured at a strict industry-standard cutoff threshold of $K = 10$, isolating the critical first-page payload evaluated by a technical recruitment team. Three foundational information retrieval metrics were computed:

Table 4.1: Retrieval Performance Comparison

Retrieval Configuration	Precision@k	MRR	NDCG@k
Word2Vec	10.00%	33.33%	11.00%
SBERT	70.00%	100.00%	79.17%
Hybrid Ensemble	70.00%	100.00%	79.36%

4.1 Precision@k

Precision@k measures the proportion of relevant rankings within the top k results returned by the system. In this project, k is set to 10. A recommendation is considered relevant if it achieves a similarity score above a defined threshold of 0.60 based on lexical and contextual semantic alignment.

The formula for Precision@k is defined as:

$$\text{Precision@K} = \frac{\sum_{i=1}^K \text{rel}_i}{K}. \quad (4.1)$$

where $\text{rel}_i \in \{0, 1\}$.

In the live system outputs, both the SBERT and Hybrid architectures achieved an optimal precision of 70.00%, accurately isolating exactly 7 true positive “Python Developer” profiles out of the top 10 returned assets despite zero explicit keyword matches. Conversely, the locally trained Word2Vec pipeline experienced severe lexical degradation, yielding a critical precision of 10.00% due to surfacing only a single relevant candidate across the top 10 positions, while the remaining 9 ranks were populated by false positives from conflicting engineering backgrounds.

4.2 Mean Reciprocal Rank (MRR)

Measures the system’s ability to surface a true match immediately without forcing the user to scroll down. It calculates the reciprocal rank of the first relevant candidate:

$$\text{MRR} = \frac{1}{\min_i(i \cdot \text{rel}_i)}. \quad (4.2)$$

An optimal score of 1.0 indicates that a true positive candidate was positioned exactly at Rank 1.

The SBERT and Hybrid Ensemble configurations demonstrated exceptional immediate retrieval accuracy, producing an ideal MRR of 100.00% ($\frac{1}{1}$) by positioning a valid true positive “Python Developer” profile at the absolute peak of the results pane (Rank 1). In

contrast, the static Word2Vec model failed to detect any relevant profiles within the top three positions, delaying its first true positive match until Rank 4 and resulting in a heavily suppressed MRR of 33.33% ($\frac{1}{3}$).

4.3 Normalized Discounted Cumulative Gain (NDCG@k)

Evaluates the global sorting quality of the ranking pipeline. It applies a logarithmic position-based discount to penalize the system if relevant candidates are pushed toward the bottom of the top-10 stack, normalizing the result against an ideal configuration (IDCG):

$$DCG_K = \sum_{i=1}^K \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)} \implies \text{NDCG}_K = \frac{DCG_K}{IDCG_K}. \quad (4.3)$$

In our system’s output, the sorting evaluation proved optimal for the standalone SBERT pipeline with an NDCG@10 of 79.17% driven by a dense clustering of true positive records at the highest tier of the stack (Ranks 1, 2, and 3), minimizing positional discounts. The Hybrid Ensemble achieved a comparable NDCG@10 of 79.36%, matching SBERT’s overall relevance extraction capacity but introducing a minor structural variation shift of +0.19% due to local keyword weights causing a minor rank swap between two higher-tier relevant profiles. Meanwhile, the Word2Vec configuration failed with a nominal NDCG@10 of 11.00% because its single true positive hit was heavily penalized at Rank 4 with no supplementary hits downstream.

4.4 Cosine Similarity Score

Cosine similarity is used to measure the similarity between the resume vector and the job description vector. The score ranges between 0 and 1, where:

- 0 $\rightarrow$ No similarity
- 1 $\rightarrow$ Perfect similarity

In the system, scores equal to or greater than 0.70 are categorized as “Best Match,” while scores between 0.40 and 0.69 are categorized as “Potential Match”. This categorization helps the frontend provide visual feedback to users regarding the strength of the recommendation.

4.5 Technical Analysis & Interpretation

- The catastrophic failure of the Word2Vec configuration is fundamentally attributed to lexical sparsity and out-of-vocabulary constraints. Training static word embedding models from scratch on highly constrained, domain-specific corpora (where $N =$

1429) introduces severe semantic drift. In the absence of the explicit anchor token “Python”, the underlying mean token vector pooling mechanism results in centroid vector dilution. This geometric shift skews the composite query representation toward a generic web-developer archetype, leaving the model mathematically incapable of inferring the latent taxonomic relationship between the framework token “Django” and its parent ecosystem.

- Conversely, the robust performance of the Sentence-BERT (SBERT) architecture underscores the efficacy of transformer-based contextualization. By leveraging multi-head self-attention mechanisms pre-trained on large-scale textual corpora, the model inherently bypasses the traditional vocabulary mismatch problem. The underlying encoding framework maps and aligns explicit conceptual tokens (e.g., “Django”, “ORM”) to the latent semantic profile of a “Python Developer” within a dense vector space. This spatial configuration preserves semantic topology, thereby facilitating robust, zero-shot concept matching without requiring explicit keyword overlap.
- Finally, the Hybrid Ensemble formulation establishes an operational equilibrium designed for production-grade pipeline stabilization. By defining the composite scoring function as a weighted linear combination ($0.60 \cdot S_{\text{SBERT}} + 0.40 \cdot S_{\text{W2V}}$), the architecture leverages SBERT as a contextual abstraction layer to mitigate local token omissions. Concurrently, retaining the secondary static word embedding layer is vital for industrial deployment, when domain specialists query using highly localized acronyms, internal tool codes, or precise version strings, the token-level layer keeps the search spaces tightly anchored, effectively suppressing semantic drift.

4.6 Overall Evaluation

For a live corporate deployment, relying exclusively on deep transformer models creates a vulnerability to semantic drift when specific, exact terms are required. Conversely, relying strictly on static localized embeddings leads to catastrophic retrieval failure when keywords vary. The Hybrid Ensemble successfully resolves this dichotomy, delivering a robust, error-correcting search pipeline tailored for practical human resource matching systems.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project developed a context-aware, hybrid resume ranking framework to address the limitations of traditional keyword-based Applicant Tracking Systems. Two independent embedding pipelines, S-BERT using all-MiniLM-L6-v2 and a Skip-Gram Word2Vec model, were implemented and evaluated on a corpus of approximately 1,429 resumes across 23 domains. Under an adversarial test that withheld the keyword "Python" from a "Django Backend Developer" query, S-BERT achieved a Precision@10 of 70.00%, an MRR of 100.00%, and an NDCG@10 of 79.17%, while Word2Vec, limited by lexical sparsity, achieved only 10.00% Precision@10 and 33.33% MRR. The preliminary Hybrid Ensemble matched or slightly exceeded standalone S-BERT performance, confirming that combining contextual and lexical embeddings yields a more reliable ranking system than either approach alone.

5.2 Future Work

Future work should focus on several directions to extend and strengthen the present system. The current 0.60/0.40 hybrid weighting between S-BERT and Word2Vec scores was chosen empirically and should be replaced with a systematically tuned or learned scheme validated against a larger annotated dataset, with evaluation broadened beyond a single adversarial query to cover multiple job descriptions across all 23 domains so that reported gains generalize rather than reflect one favorable test case. Incorporating TF-IDF as a third ranking signal may further strengthen matching for rare acronyms or product-specific terminology that embedding models underrepresent.

The ranking pipeline is also planned for full integration into an interactive web application capable of returning color-coded, real-time results to HR users, with approximate nearest-neighbor indexing techniques such as FAISS explored to preserve low query latency as the resume corpus grows. Finally, since preprocessing already strips PERSON, GPE, and LOC entities to limit identity-based bias, future work will include a fairness audit of ranking outputs alongside a feedback mechanism allowing HR users to mark candidates as relevant or irrelevant for continuous model refinement.

REFERENCES

- [1] V. Mittal, P. Mehta, D. Relan, and G. Gabrani, “Methodology for resume parsing and job domain prediction,” *Journal of Statistics and Management Systems*, vol. 23, no. 7, pp. 1265–1274, 2020.
- [2] P. K. Roy, S. S. Chowdhary, and R. Bhatia, “A machine learning approach for automation of resume recommendation system,” *Procedia Computer Science*, vol. 167, pp. 2318–2327, 2020.
- [3] A. Joseph, J. Sunny, R. Raveena, K. Laya, *et al.*, “Resume analyser: Automated resume ranking software,” *International Journal for Research in Applied Science and Engineering Technology*, vol. 8, pp. 896–899, 2020.
- [4] R. Ransing, A. Mohan, N. B. Emberi, and K. Mahavarkar, “Screening and ranking resumes using stacked model,” in *2021 5th International Conference on Electrical, Electronics, Communication, Computer Technologies and Optimization Techniques (ICEECCOT)*, pp. 643–648, IEEE, 2021.
- [5] N. Tülümen, G. Akgün, A. Nohutçu, G. S. A. Genç, and S. Genç, “Hybrid job and resume matcher,” in *2021 6th International Conference on Computer Science and Engineering (UBMK)*, pp. 163–168, IEEE, 2021.
- [6] K. Tejaswini, V. Umadevi, M. K. Shashank, *et al.*, “Design and development of machine learning based resume ranking system,” *Global Transitions Proceedings*, vol. 3, no. 2, pp. 371–375, 2022.
- [7] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” *arXiv preprint arXiv:1301.3781*, 2013.
- [8] N. Bhoir, M. Jakate, S. Lavangare, A. Das, and S. Kolhe, “Resume parser using hybrid approach to enhance the efficiency of automated recruitment processes,” *Authorea Preprints*, 2023.
- [9] A. Widiyanto, E. Pebriyanto, F. Fitriyanti, and M. Marna, “Document similarity using term frequency-inverse document frequency representation and cosine similarity,” *Journal of Dinda: Data Science, Information Technology, and Data Analytics*, vol. 4, no. 2, pp. 149–153, 2024.
- [10] V. S. Tallapragada, V. S. Raj, U. Deepak, P. D. Sai, and T. Mallikarjuna, “Improved resume parsing based on contextual meaning extraction using bert,” in *2023 7th International Conference on Intelligent Computing and Control Systems (ICICCS)*, pp. 1702–1708, IEEE, 2023.

- [11] V. Jagwani, S. Meghani, K. Pai, and S. Dhage, “Resume evaluation through latent dirichlet allocation and natural language processing for effective candidate selection,” *arXiv preprint arXiv:2307.15752*, 2023.
- [12] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” in *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pp. 3982–3992, 2019.
- [13] C. Onukwugha, C. Ofoegbu, O. Aliche, and C. Bertrand, “Resume optimization model using machine learning techniques,” *International Journal of Intelligent Information Systems*, vol. 13, no. 5, pp. 109–116, 2024.
- [14] O. E. Olorunshola, I. O. Ampitan, F. Adamu-Fika, and A. K. Ademuwagun, “An enhanced k-nn algorithm leveraging bert techniques for resume parsing system,” *Asian Journal of Research in Computer Science*, vol. 18, no. 7, pp. 49–59, 2025.
- [15] R. Sarveshwaran, S. Karthikeyan, M. V. Cruz, S. Shreyanth, S. Niveditha, and P. Rajesh, “Nlp-based ai-driven resume screening solution for efficient candidate selection,” in *International Congress on Information and Communication Technology*, pp. 359–370, Springer, 2024.
- [16] “Resume dataset.” <https://www.kaggle.com/datasets/snehaanbhawal/resume-dataset>, 2026.

APPENDIX A Word2Vec Word Cloud

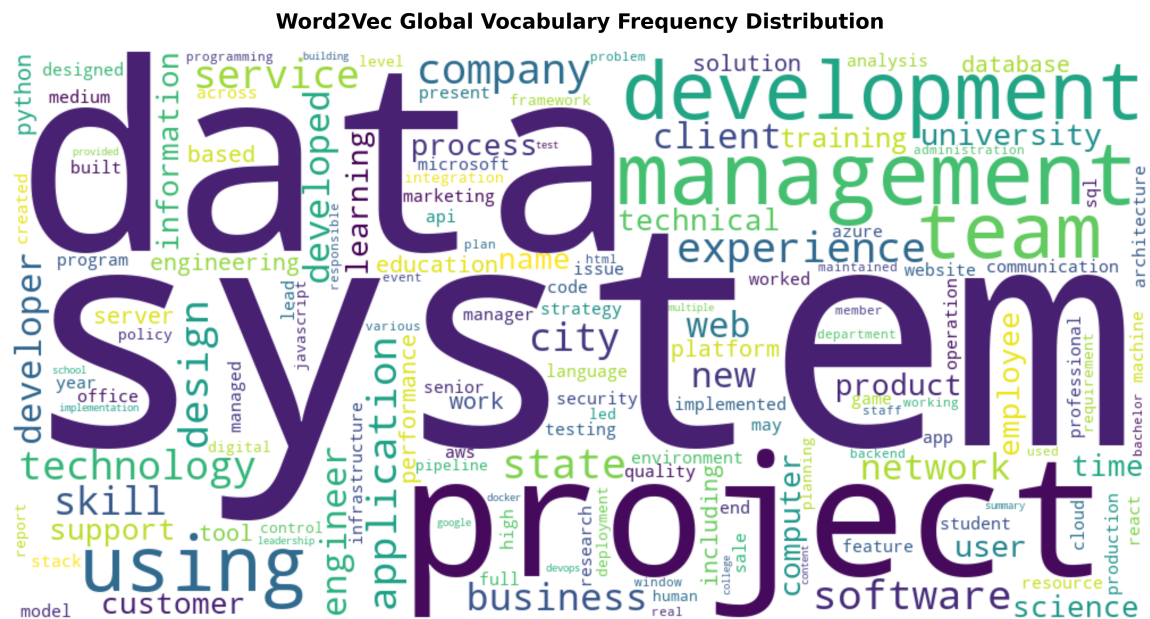


Figure A.1: Word2Vec Wordcloud

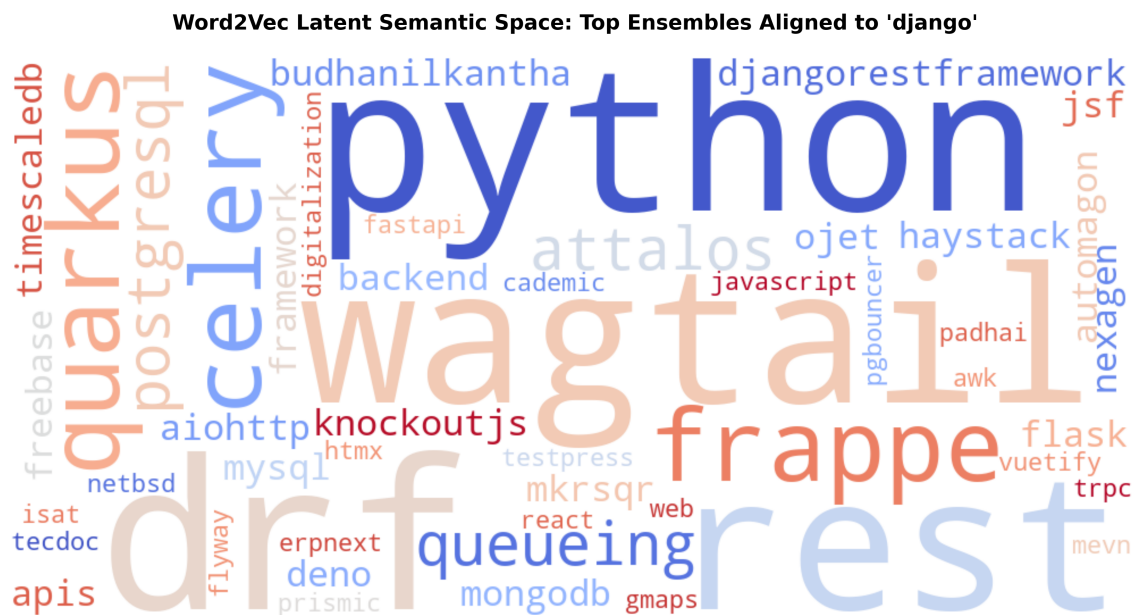


Figure A.2: Word Cloud for Django Resume

APPENDIX B Web UI Images

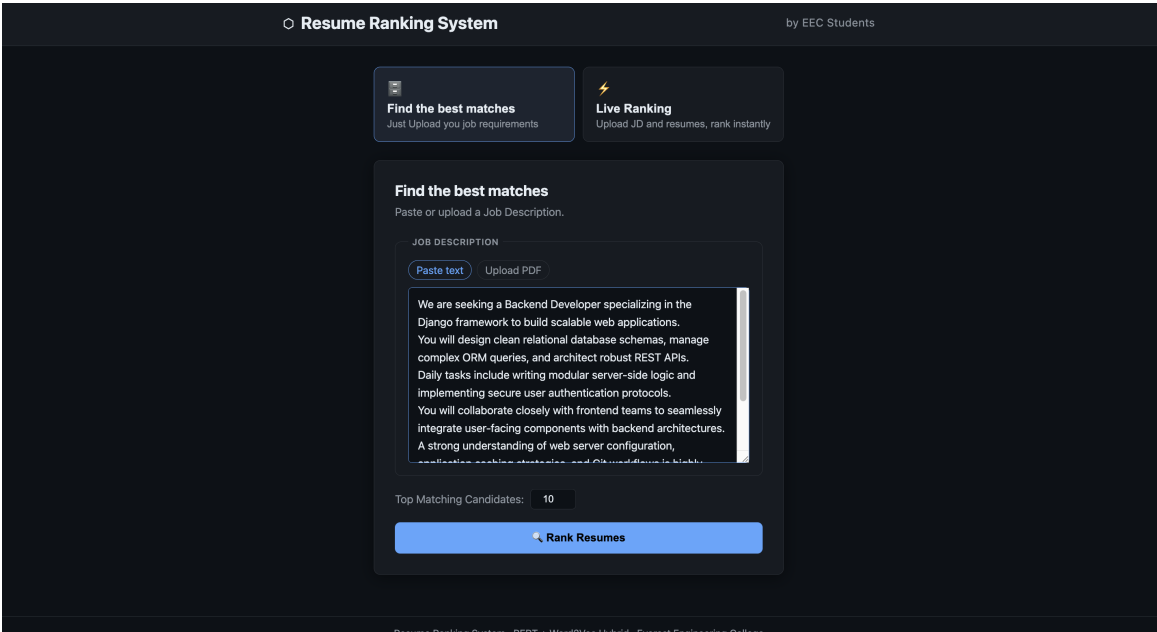


Figure B.1: Providing JD Text

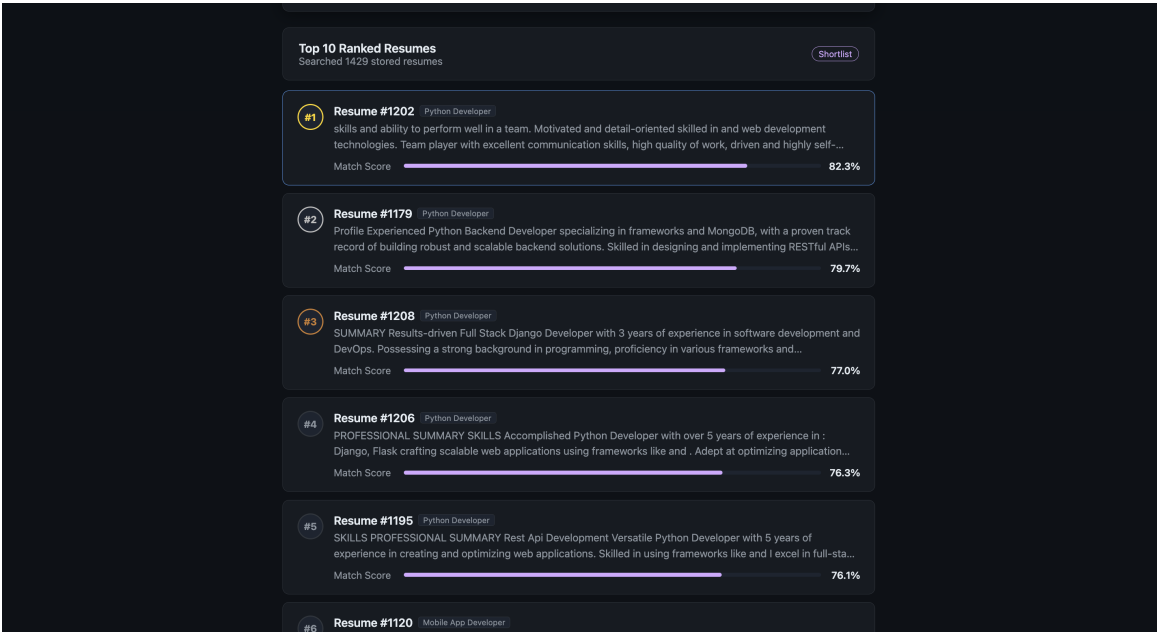


Figure B.2: Ranked Available Resumes For Given JD Text

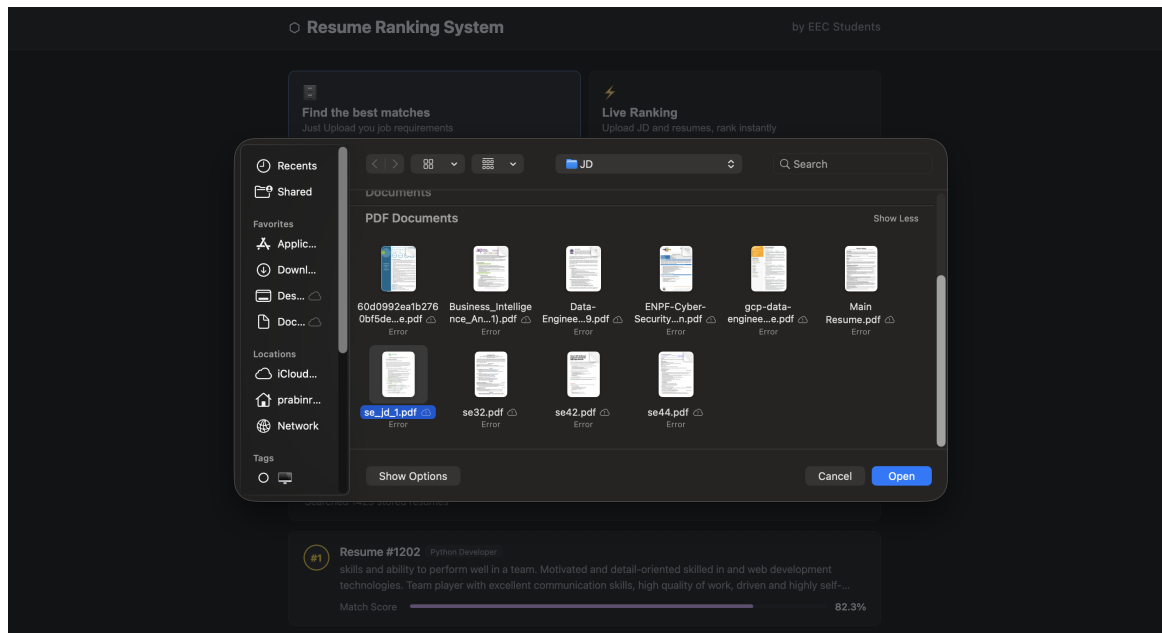


Figure B.3: Uploading PDFs

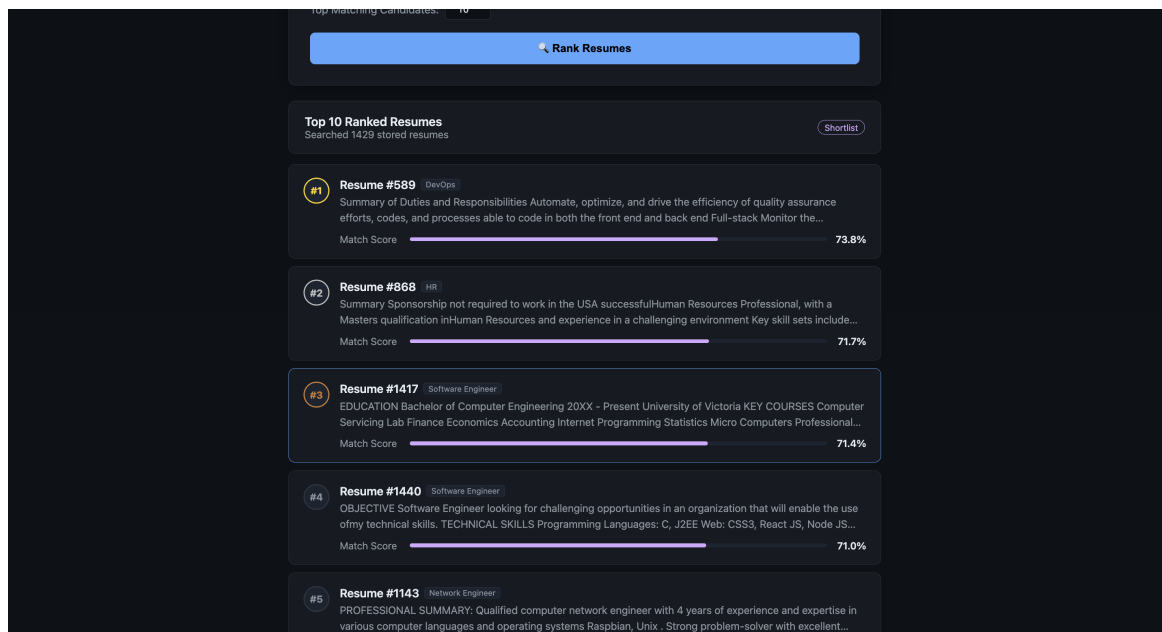


Figure B.4: Ranked Available Resumes For Given JD PDF

○ Resume Ranking System

by EEC Students

Find the best matches

Just Upload you job requirements

Live Ranking

Upload JD and resumes, rank instantly

Live Ranking

Upload a Job Description and multiple Resume PDFs to rank them instantly.

JOB DESCRIPTION

Paste text

Upload PDF

Paste job description here ...

RESUME PDFS

files

Click to choose resume PDFs

Top Matching Candidates:

10

✦ Extract & Rank

Figure B.5: Candidate Sorting

4

APPENDIX C Metric Performance Images

— SYSTEM PERFORMANCE MATRIX (Target Domain: 'Python Developer') —

Retrieval Strategy	Precision@10	MRR	NDCG@10
Word2Vec Local	10.00%	33.33%	11.00%
SBERT Semantic	70.00%	100.00%	79.17%
Hybrid Ensemble	70.00%	100.00%	79.36%

Figure C.1: Performance Metrics